

Концепт: Decision Tree & Random Forest — Классификация вин

Этот файл объясняет каждый блок ноутбука `Classification-for-task4_part_1.ipynb` — что делается, зачем и как это работает под капотом.

Оглавление

1. [Импорты](#)
2. [Загрузка данных и EDA](#)
3. [Разделение данных](#)
4. [Decision Tree — подбор параметров \(criterion, max_features, max_depth\)](#)
5. [Визуализация дерева: gini vs entropy](#)
6. [Почему gini высокий в корне и стремится к 0 на листьях](#)
7. [Decision Tree — оценка модели](#)
8. [ROC-кривая и оптимальный порог](#)
9. [Random Forest — подбор числа деревьев](#)
10. [Random Forest — оценка модели](#)
11. [Важность признаков](#)
12. [Precision vs Recall](#)
13. [Предсказание нового вина](#)

1. Импорты

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, confusion_matrix, classification_report,
                             ConfusionMatrixDisplay, roc_curve, auc)
from sklearn.preprocessing import label_binarize
```

Библиотека	Зачем
numpy	Математика с массивами (argmax, argsort и т.д.)
matplotlib / seaborn	Графики
pandas	Таблицы, датафреймы
load_wine	Встроенный датасет с винами из sklearn
train_test_split	Разделение данных на обучение и тест
GridSearchCV	Автоматический перебор параметров с кросс-валидацией
DecisionTreeClassifier	Модель дерева решений
plot_tree	Визуализация структуры дерева решений
RandomForestClassifier	Модель случайного леса
accuracy_score	Доля правильных предсказаний
confusion_matrix	Матрица ошибок
classification_report	Precision, Recall, F1 по каждому классу
roc_curve , auc	ROC-кривая и площадь под ней
label_binarize	Перевод меток в формат one-vs-rest для ROC

Почему нет StandardScaler?

Деревья решений и случайный лес не зависят от масштаба признаков.

Они делят данные по пороговым значениям (например, `alcohol > 13.5`), а не вычисляют расстояния. Масштабирование им не нужно.

2. Загрузка данных и EDA

Загрузка

```
df = load_wine(as_frame=True)
df = df.frame
```

- `load_wine()` — загружает встроенный датасет: 178 вин, 13 химических признаков, 3 сорта (классы 0, 1, 2).
- `as_frame=True` — возвращает объект Bunch, где данные уже в виде DataFrame.
- `.frame` — берём именно DataFrame (признаки + колонка `target` вместе).

Первичный осмотр

```
df.head(70)      # первые 70 строк — просто глазами проверяем данные
df.info()         # типы данных, количество непустых значений
df.describe()     # мин/макс/среднее/std по каждому признаку
df['target'].value_counts() # сколько образцов в каждом классе
```

Что смотрим:

- `info()` — нет ли пропущенных значений (Non-Null Count должен быть 178 везде). У нас всё чисто.
- `describe()` — нет ли аномалий (например, `proline` идёт от 278 до 1680 — большой разброс, но для дерева это не проблема).
- `value_counts()` — баланс классов: класс 1 = 71, класс 0 = 59, класс 2 = 48. Дисбаланс умеренный.

Корреляционная тепловая карта

```
sns.heatmap(df.corr(), annot=True, fmt='.1f', cmap='coolwarm')
```

- `df.corr()` — матрица корреляций всех признаков между собой (значения от -1 до 1).
- `annot=True` — показывает числа на карте.
- `cmap='coolwarm'` — синий = отрицательная корреляция, красный = положительная.

Зачем: видим какие признаки связаны. Например, если `flavanoids` и `total_phenols` сильно коррелируют — они несут похожую информацию. Дерево само разберётся что важнее, но нам полезно понимать структуру данных.

Боксплоты

```
sns.boxplot(x='target', y='alcohol', data=df)
```

Показывает распределение `alcohol` отдельно для каждого класса вина.
Если боксы плохо пересекаются — признак хорошо разделяет классы.

```
features = df.columns.drop('target')
n_cols = 3
n_rows = (len(features) + n_cols - 1) // n_cols # сколько строк нужно (13 / 3 = 5 строк)
fig, axes = plt.subplots(n_rows, n_cols, ...)
axes = axes.flatten() # превращаем 2D-массив осей в 1D для удобного перебора

for i, col in enumerate(features):
    sns.boxplot(..., ax=axes[i]) # рисуем каждый признак в свою ячейку

for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j]) # удаляем лишние пустые ячейки (13 признаков, а ячеек 15)
```

Итог EDA: понимаем данные — нет пропусков, умеренный дисбаланс классов, некоторые признаки явно разделяют классы (например, `proline`, `flavanoids`).

3. Разделение данных

```
X = df.drop('target', axis=1) # признаки (13 колонок)
y = df['target']              # метки классов (0, 1, 2)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- `test_size=0.2` — 20% данных (36 строк) идут в тест, 80% (142 строки) на обучение.
- `random_state=42` — фиксируем случайность, чтобы каждый раз было одинаковое разбиение.

Правило: модель обучается только на `X_train / y_train`. `X_test / y_test` — "нетронутые" данные, на которых мы честно проверяем качество в конце.

4. Decision Tree — подбор параметров

Что такое Decision Tree (дерево решений)

Дерево решений — это алгоритм, который строит цепочку вопросов-условий:

```
alcohol > 13.5?
├─ Да → flavanoids > 2.8? → ...
└─ Нет → proline < 600? → ...
```

Каждый вопрос делит данные на две части. На конце ветки — предсказание класса.

GridSearchCV — три параметра одновременно

```
param_grid_dt = {
    'max_depth': list(range(1, 15)),
    'criterion': ['gini', 'entropy'],
    'max_features': [None, 'sqrt', 'log2']
}
```

Итого: $14 \times 2 \times 3 = 84$ комбинации $\times$ 5 фолдов = 420 обучений. GridSearchCV пробует все и выбирает лучшую.

max_depth — максимальная глубина дерева:

- Маленькая (1-2) → недообучение, слишком простые правила.
- Очень большая → дерево заучивает обучающие данные, переобучение.
- Нужно найти золотую середину.

criterion — чем измерять "нечистоту" узла при выборе разбиения:

- 'gini' — индекс Джини (по умолчанию). Вычисляется быстрее.

- 'entropy' — информационный выигрыш (энтропия). Математически чище.
- На практике дают схожий результат; GridSearchCV сам выберет лучшее.

max_features — сколько признаков рассматривать в каждом узле при поиске лучшего разбиения:

Значение	Сколько признаков	Пример (13 признаков)
None	Все признаки	13 из 13
'sqrt'	$\sqrt{n_features}$	$\sqrt{13} \approx 3-4$
'log2'	$\log_2(n_features)$	$\log_2(13) \approx 3$

Зачем ограничивать? При None дерево всегда найдёт лучший признак → более "жадное" дерево, склонное к переобучению.

'sqrt' и 'log2' добавляют случайность — дерево рассматривает только подмножество признаков в каждом узле. Это снижает переобучение. Именно поэтому Random Forest по умолчанию использует 'sqrt'.

```
grid_dt = GridSearchCV(DecisionTreeClassifier(random_state=42), param_grid_dt, cv=5, scoring='accuracy')
grid_dt.fit(X_train, y_train)
```

```
print(f"Лучшие параметры: {grid_dt.best_params_}")
```

Как работает GridSearchCV:

1. Берёт каждую из 84 комбинаций параметров.
2. Делит `x_train` на 5 фолдов, обучает и проверяет 5 раз.
3. Усредняет точность.
4. Выбирает комбинацию с наилучшим средним результатом.

```
dt = grid_dt.best_estimator_ # лучшая уже обученная модель
dt_pred = dt.predict(X_test)
```

`best_estimator_` — GridSearchCV сам переобучил победившую комбинацию на всём `x_train`.

5. Визуализация дерева: gini vs entropy

```
from sklearn.tree import plot_tree

fig, axes = plt.subplots(1, 2, figsize=(28, 10))

for ax, criterion in zip(axes, ['gini', 'entropy']):
    dt_vis = DecisionTreeClassifier(max_depth=3, criterion=criterion, random_state=42)
    dt_vis.fit(X_train, y_train)
    plot_tree(dt_vis,
              feature_names=list(X.columns),
              class_names=target_names,
              filled=True,
              rounded=True,
              fontsize=9,
              ax=ax)
    ax.set_title(f'criterion={criterion}')
```

Что видно на картинке:

- Каждый прямоугольник = один узел дерева.
- Верхняя строка — условие разбиения (`flavanoids <= 1.575`).
- `gini` или `entropy` — значение нечистоты этого узла (см. раздел 6).
- `samples` — сколько обучающих примеров попало в этот узел.
- `value` — сколько примеров каждого класса: `[class_0, class_1, class_2]` .
- `class` — предсказание, если остановиться здесь (мажоритарный класс).
- **Цвет** — чем насыщеннее, тем чище узел (меньше примесей других классов).

`filled=True` — окрашивает узел в цвет доминирующего класса.

`rounded=True` — скруглённые углы (чисто визуально).

6. Почему gini высокий в корне и стремится к 0 на листьях

Индекс Джини (Gini Impurity)

Формула: $Gini = 1 - \sum(p_i^2)$

где p_i — доля класса i в узле.

Пример — корневой узел (все 142 обучающих примера, три класса примерно поровну):

- $\text{class_0: } 47/142 \approx 0.33$, $\text{class_1: } 57/142 \approx 0.40$, $\text{class_2: } 38/142 \approx 0.27$
- $\text{Gini} = 1 - (0.33^2 + 0.40^2 + 0.27^2) = 1 - (0.109 + 0.160 + 0.073) = \approx \mathbf{0.66}$

Высокое значение — узел "грязный", три класса перемешаны.

Пример — чистый лист (все 38 примеров — только class_2):

- $\text{class_2: } 38/38 = 1.0$, остальные = 0
- $\text{Gini} = 1 - (0^2 + 0^2 + 1^2) = 1 - 1 = \mathbf{0.0}$

Нулевое значение — узел "чистый", один класс.

Вывод: $\text{Gini} = 0$ означает идеальное разделение. Дерево при каждом разбиении выбирает признак и порог, которые **максимально снижают** Gini — это и есть критерий качества разбиения.

Энтропия (Entropy)

Формула: $\text{Entropy} = -\sum(p_i \cdot \log_2(p_i))$

- Максимум при равном распределении классов (в корне).
- Минимум (0) при чистом узле (на листе).

Разница между gini и entropy:

- Gini немного быстрее вычисляется.
- Entropy чуть сильнее штрафует "почти чистые, но не совсем" узлы.
- На практике разница в точности минимальная — `GridSearchCV` выберет лучшее.

7. Decision Tree — оценка модели

Classification Report

```
print(classification_report(y_test, dt_pred))
```

Выводит таблицу:

	Precision	Recall	F1-score	Support
0	0.92	1.00	0.96	14
1	1.00	0.93	0.96	14
2	1.00	1.00	1.00	8

- **Precision** — из всех кого модель назвала "классом X", сколько реально класс X.
 - Пример: модель сказала "класс 0" 10 раз → 9 из них правда класс 0 → Precision = 0.9
- **Recall** — из всех реальных "класс X", сколько модель нашла.
 - Пример: реальных класс 0 было 14 → нашла 12 → Recall = 0.86
- **F1-score** — среднее гармоническое Precision и Recall. Один итоговый показатель.
- **Support** — сколько образцов этого класса в тест-выборке.

Confusion Matrix (Матрица ошибок)

```
cm = confusion_matrix(y_test, dt_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues')
```

Матрица показывает:

- По строкам — реальные классы.
- По столбцам — предсказанные классы.
- Главная диагональ = правильные предсказания.
- Всё вне диагонали = ошибки.

```

      Предсказано:
      0      1      2
Реально: 0 [14   0   0]  ← все 14 вина класса 0 угаданы верно
          1 [ 0  13   1]  ← 1 вино класса 1 ошибочно названо классом 2
          2 [ 0   0   8]  ← все 8 вин класса 2 угаданы верно
```

8. ROC-кривая и оптимальный порог

Зачем нужна ROC при multiclass?

У нас 3 класса, а ROC строится для бинарных задач. Поэтому используем подход **One-vs-Rest (OvR)**: отдельно строим ROC для каждого класса (класс X vs все остальные).

```
y_test_bin = label_binarize(y_test, classes=[0, 1, 2])
```

Превращает [0, 1, 2, 1, 0, ...] в матрицу:

```
[[1, 0, 0], # это класс 0
 [0, 1, 0], # это класс 1
 [0, 0, 1], # это класс 2
 ...]
```

```
y_score_dt = dt.predict_proba(X_test)
```

- `predict_proba()` возвращает вероятности для каждого класса.
- Пример: [0.8, 0.1, 0.1] — модель на 80% уверена, что это класс 0.

```
for i in range(3):
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_score_dt[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f'Класс {i} (AUC = {roc_auc:.2f})')
```

- `fpr` (False Positive Rate) — доля ложных срабатываний.
- `tpr` (True Positive Rate = Recall) — доля правильно найденных.
- **AUC** (площадь под кривой) — чем ближе к 1.0, тем лучше модель. 0.5 = случайное угадывание.

```
plt.plot([0, 1], [0, 1], 'k--') # пунктирная линия = случайная модель (AUC=0.5)
```

Оптимальный порог (метод Юдена)

```
for i in range(3):
    fpr, tpr, thresholds = roc_curve(y_test_bin[:, i], y_score_dt[:, i])
    j_scores = tpr - fpr          # Индекс Юдена = TPR - FPR
    best_idx = np.argmax(j_scores) # Где максимум — там лучший порог
    print(f"Класс {i}: порог = {thresholds[best_idx]:.2f}")
```

Зачем менять порог?

По умолчанию модель предсказывает класс если вероятность > 0.5 . Но иногда оптимально поставить порог 0.3 или 0.7.

Метод Юдена: ищем порог где $TPR - FPR$ максимален — т.е. модель много угадывает правильных и мало ошибается. Это геометрически точка на ROC-кривой, ближайшая к верхнему левому углу.

9. Random Forest — подбор числа деревьев

Что такое Random Forest (случайный лес)

Случайный лес — это ансамбль из многих деревьев решений:

1. Берёт случайную подвыборку данных (с возвращением — bootstrap).
2. Строит дерево, используя только случайное подмножество признаков в каждом разбиении.
3. Повторяет N раз ($N = n_estimators$).
4. Итоговое предсказание — **голосование**: какой класс назвало большинство деревьев.

Почему это лучше одного дерева:

- Каждое дерево видит разные данные → разные ошибки.
- Усредняя много "немного разных" деревьев, мы уменьшаем разброс (variance).
- Одно дерево легко переобучается; лес — намного устойчивее.

GridSearchCV для RF

```
param_grid_rf = {'n_estimators': [10, 20, 50, 100, 150, 200]}

grid_rf = GridSearchCV(RandomForestClassifier(random_state=42), param_grid_rf, cv=5, scoring='a
grid_rf.fit(X_train, y_train)
```

Логика та же, что для дерева — перебираем число деревьев, выбираем лучшее по CV.

Обычно точность растёт до ~100 деревьев и дальше почти не меняется, только замедляется обучение.

```
rf = grid_rf.best_estimator_ # лучшая RF-модель
rf_pred = rf.predict(X_test)
```

10. Random Forest — оценка модели

Те же метрики, что для Decision Tree — confusion matrix, classification report, ROC.

```
y_score_rf = rf.predict_proba(X_test) # вероятности от RF
```

RF возвращает усреднённые вероятности по всем деревьям — как правило более "уверенные" и точные, чем у одного дерева.

11. Важность признаков

```
importances = rf.feature_importances_
```

После обучения RF каждому признаку назначается "важность" — насколько сильно он уменьшал ошибку при разбиениях во всех деревьях. Сумма всех важностей = 1.0.

```
indices = np.argsort(importances)[::-1] # сортируем от большего к меньшему
```

- `np.argsort()` возвращает индексы, отсортированные по возрастанию.
- `[::-1]` разворачивает → получаем убывающий порядок.

```
plt.bar(range(len(importances)), importances[indices])
plt.xticks(range(len(importances)), feature_names[indices], rotation=45)
```

Строим столбчатый график: самый важный признак слева, наименее важный справа.

Почему RF, а не DT для важности признаков?

RF усредняет важности по сотням деревьев → оценка стабильнее и надёжнее, чем у одного дерева.

12. Precision vs Recall

Precision (точность) — "когда говоришь X, ты прав?"

- Формула: $TP / (TP + FP)$
- Ложноположительная ошибка (FP): сказал "класс 0", а это класс 1.

Recall (полнота) — "всех ли X ты нашёл?"

- Формула: $TP / (TP + FN)$
- Ложноотрицательная ошибка (FN): не распознал класс 0, назвал его классом 1.

Для классификации вин важнее Precision, потому что:

- Неправильно определить сорт вина = неверная маркировка продукта.
- Это прямая ошибка в продукте (юридическая, коммерческая).
- Лучше иногда сказать "не уверен" (низкий Recall), чем уверенно ошибиться (низкий Precision).

13. Предсказание нового вина

```
new_wine_data = [13.7, 1.8, 2.3, 16.0, 110.0, 2.5, 2.3, 0.3, 1.8, 5.2, 1.1, 3.2, 1000.0]
```

13 значений признаков — одно "новое" вино, которого не было в датасете.

```
new_wine_df = pd.DataFrame([new_wine_data], columns=X.columns)
```

Оборачиваем в DataFrame с правильными именами колонок — sklearn ожидает именно такой формат (чтобы не было предупреждений о feature names).

```
dt_pred_new = dt.predict(new_wine_df)
dt_proba_new = dt.predict_proba(new_wine_df)
```

```
rf_pred_new = rf.predict(new_wine_df)
rf_proba_new = rf.predict_proba(new_wine_df)
```

- `predict()` — класс с наибольшей вероятностью.
- `predict_proba()` — вероятности по всем трём классам.

```
target_names = load_wine().target_names # ['class_0', 'class_1', 'class_2']
print(f"Decision Tree — сорт: {target_names[dt_pred_new[0]]}, вероятности: {dt_proba_new[0]}")
print(f"Random Forest — сорт: {target_names[rf_pred_new[0]]}, вероятности: {rf_proba_new[0]}")
```

Сравниваем результаты двух моделей на одном и том же новом примере.

Итоговая схема пайплайна

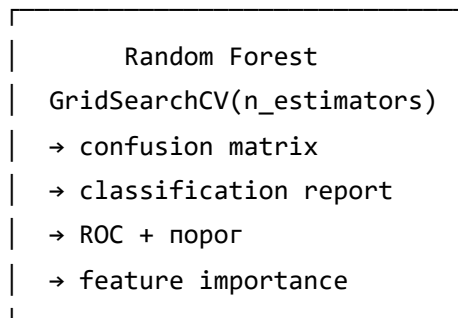
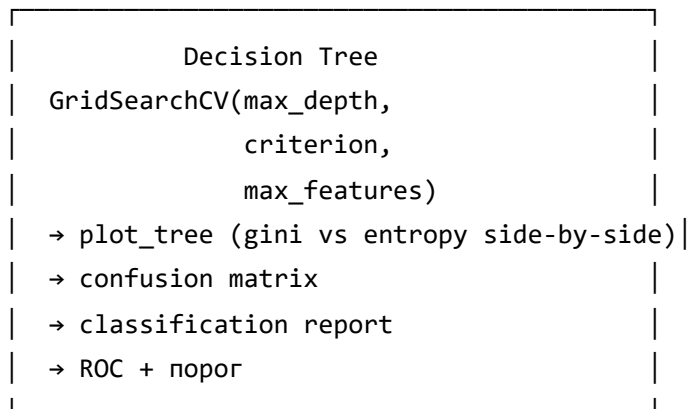
Данные (178 вин, 13 признаков, 3 класса)



EDA (info, describe, heatmap, boxplots)



train_test_split (80% / 20%)



Вывод: Precision важнее Recall



Предсказание нового вина